



AG-GNN: Adaptive gating mechanism for robust node classification in graph neural networks

Ahmed Begga^{*} , Miguel Ángel Lozano , Francisco Escolano

Department of Computer Science and Artificial Intelligence, University of Alicante, Alicante, Spain

HIGHLIGHTS

- Our model dynamically balances node features and graph structure through an adaptive gating mechanism.
- A novel multi-pathway architecture automatically handles both homophilic and heterophilic graphs.
- The proposed model maintains stable performance across network depths without over-smoothing.
- Extensive experiments on 15 datasets demonstrate superior performance and scalability.
- The model effectively scales to graphs with millions of nodes where baseline models fail.

ARTICLE INFO

Keywords:

Node classification
Graph neural networks
Homophily and heterophily
Over-smoothing
Complex network

ABSTRACT

Graph Neural Networks (GNNs) have revolutionized node classification tasks by leveraging graph structure and node features through message-passing schemes. However, GNNs frequently suffer from over-smoothing as the number of layers increases, causing node representations to collapse and lose discriminative power. In this paper, we propose AG-GNN, a novel architecture that addresses these challenges through a simple yet effective adaptive gating mechanism. This mechanism acts as a smart switch that dynamically controls how much information flows from the graph structure versus the node features at each layer. Our model's dual-pathway design enables it to excel in both homophilic graphs (where connected nodes tend to share the same class) and heterophilic graphs (where connected nodes often belong to different classes). Extensive experiments demonstrate that AG-GNN consistently outperforms state-of-the-art methods, achieving up to 2.16 % improvement on heterophilic datasets like Cornell and 5.86 % improvement on large-scale networks like Penn94. Importantly, our approach maintains strong performance even with very deep architectures (up to 64 layers), demonstrating remarkable resistance to over-smoothing where traditional GNNs fail. AG-GNN scales efficiently to graphs with millions of nodes while maintaining computational tractability whereas several baseline models experience out-of-memory errors.

1. Introduction

Graph Neural Networks (GNNs) [1,2] have become the cornerstone for solving graph-related tasks [3], such as node classification [4], link prediction [5], and graph classification [6]. By capturing structural information and node features through message passing, GNNs offer a powerful framework for learning meaningful node embeddings that reflect both local and global graph properties [7]. However, as GNNs increase in depth, they often face the issue of over-smoothing [8], where repeated message passing

^{*} Corresponding author.

Email address: ahmed.begga@ua.es (A. Begga).

<https://doi.org/10.1016/j.ins.2025.122750>

Received 29 November 2024; Received in revised form 11 September 2025; Accepted 2 October 2025

Available online 3 October 2025

0020-0255/© 2025 The Author(s). Published by Elsevier Inc. This is an open access article under the CC BY license (<http://creativecommons.org/licenses/by/4.0/>).

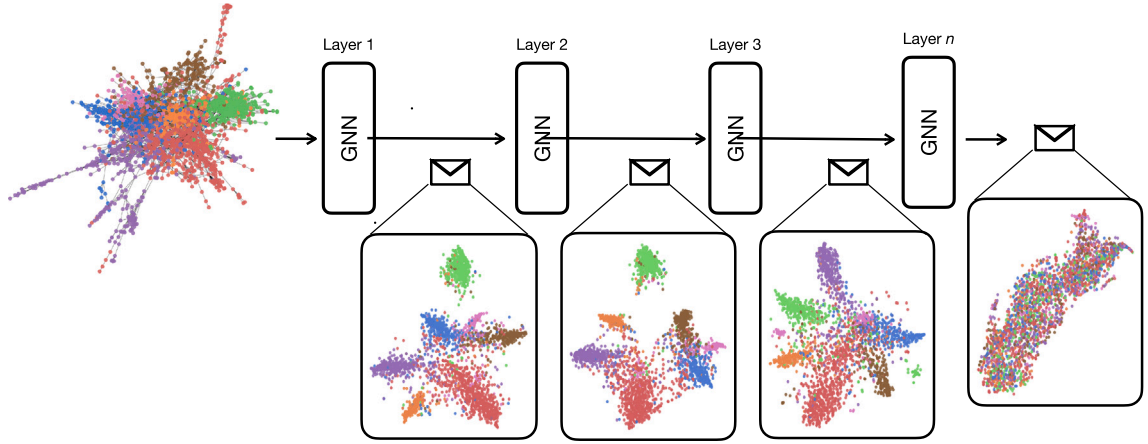


Fig. 1. Visualization of the over-smoothing phenomenon in GNNs. As layers increase from left to right, the initially well-separated node clusters (shown in different colors) progressively merge due to excessive message passing. The diagram shows how node representations evolve through successive GNN layers (Layer 1 to Layer n), demonstrating the gradual loss of discriminative power as node features become increasingly homogeneous.

causes node representations to become increasingly similar and eventually indistinguishable. As illustrated in Fig. 1, this phenomenon becomes more pronounced with each additional layer, leading to a progressive mixing of node features that ultimately results in poor classification performance [9]. This effect is particularly severe in heterophilic networks, where the over-smoothing process begins at earlier layers due to the inherent diversity of connected nodes [10].

Researchers have proposed various strategies to address the over-smoothing challenge, each with its own limitations. Regularization-based approaches [11,12] aim to slow down the convergence of node representations but merely delay rather than fundamentally prevent over-smoothing. Residual connection techniques [8,13] preserve information from earlier layers but often struggle to determine the optimal balance between original features and new structural information, particularly in heterophilic settings. Attention mechanisms [14] adaptively weight neighbor contributions but typically cannot completely ignore misleading neighborhood information when necessary. Methods that modify the graph structure or diffusion process [15,16] frequently rely on predetermined assumptions that may not generalize across diverse graph structures.

A critical gap in existing approaches is that they rarely adaptively balance the contributions of structural versus feature information based on the specific characteristics of the input graph. This limitation becomes particularly problematic when dealing with heterophilic networks—where nodes of different classes tend to connect—as existing methods often underperform due to their reliance on homophily assumptions [17]. Furthermore, many current techniques either restrict the expressivity of the model or fail to scale efficiently on larger and more complex graph structures [18]. Thus, the challenge remains to design a scalable and adaptive GNN architecture that can handle both homophilic and heterophilic settings while effectively controlling the flow of information across layers [7].

In this work, we introduce an innovative GNN architecture designed to overcome the over-smoothing problem by learning an adaptive gating mechanism. Unlike traditional message-passing schemes [19], our model learns how to selectively pass information through gates that are dynamically optimized during training. This process allows the network to surf the graph [20], dynamically balancing between retaining local node features and aggregating information from neighboring nodes, thus preserving the diversity of node representations even in deeper layers. The gating mechanism [21] effectively mitigates over-smoothing by ensuring that each node maintains a unique and informative representation [22], regardless of network depth.

Additionally, our approach is inherently scalable, as the gating mechanism reduces the computational overhead typically associated with deep GNNs. This scalability allows the model to excel not only on small benchmark datasets [4] but also on large-scale graphs [23]. Extensive empirical validation on standard datasets demonstrates that our method consistently outperforms state-of-the-art techniques, particularly in challenging heterophilic environments where existing models often struggle. Our contributions can be summarized as follows:

- We introduce a novel adaptive gating mechanism that mitigates over-smoothing and allows for very deep GNN architectures.
- We propose an adaptive feature integration mechanism that automatically balances the contribution of different information sources, effectively addressing both heterophilic and homophilic settings.
- We demonstrate scalability through experiments on large-scale graph datasets, showing superior performance compared to existing state-of-the-art methods.

In the following sections, we discuss related work, present the technical details of our model, and report extensive experimental results to validate the effectiveness of our approach.

2. Related work

GNNs have emerged as an effective approach for learning from graph-structured data, showing remarkable success across various domains [6,24,25]. However, GNN performance is often hindered by at least two key challenges: heterophily and over-smoothing [17]. Recent research has made notable strides in addressing these issues, leading to more robust and versatile GNN architectures.

Heterophily, where connected nodes are more likely to have different labels, has been a focal point of recent GNN research. Traditional GNN models, such as GCN [2] and GAT [14], rely heavily on the assumption of homophily, which limits their performance on heterophilic graphs. To overcome this limitation, Zhu et al. [17] introduced H2GCN, a model that integrates ego- and neighbor-embedding separation with higher-order neighborhoods. This design allows the model to capture more complex node relationships, significantly improving performance on heterophilic graphs.

Expanding on these ideas, Yan et al. [17] developed GGCN, a model that utilizes a generalized PageRank scheme to adapt to diverse patterns of node labels. GGCN's flexibility allows it to perform well across both homophilic and heterophilic settings, marking a significant advancement towards more generalized GNN architectures. Similarly, Chien et al. [26] proposed GPRGNN, which combines an adaptive PageRank method with GNNs to enhance feature representations, yielding further improvements on heterophilic graphs.

Recent advances in addressing heterophily have led to several innovative approaches. DAGNN [27] tackles the over-smoothing problem through a novel decoupling of transformation and propagation operations. ACM-GCN [19] introduces an Adaptive Channel Mixing framework that leverages multiple information channels, including aggregation, diversification and identity pathways, to better handle varying degrees of heterophily. CS-GNN [28] takes a different approach by introducing feature and label smoothness metrics to quantify neighborhood information quality. GloGNN [29] extends the traditional local aggregation by incorporating global node information through learned correlation coefficients. Recently, Hoseinnia et al. [30] proposed GCN + AEE + BDE, which combines Adaptive Early Embedding (AEE) and Biased DropEdge (BDE) procedures to mitigate over-smoothing through adaptive node-level decisions. This approach adaptively halts aggregation for nodes that achieve high confidence at intermediate layers, preventing excessive smoothing while removing inter-class edges to enhance information quality. These developments complement existing methods while offering new perspectives on handling heterophilic graphs.

Over-smoothing, where node representations become overly similar as GNN layers increase, is another critical issue. Li et al. [31] were among the first to identify this problem, showing that repeated application of graph convolutions results in excessive feature smoothing. This revelation has led to a series of solutions aimed at mitigating over-smoothing, allowing for very deep GNN architectures.

One common approach involves the use of residual connections. Li et al. [13] proposed ResGCN, which incorporates skip connections to mitigate over-smoothing and enable deeper network training. Building on this concept, Chen et al. [8] introduced GCNII, which combines initial residual and identity mapping techniques, allowing for the training of significantly deeper GNNs without sacrificing node distinguishability. These methods have been highly effective in preserving node uniqueness across many layers.

Recently, theoretical foundations for understanding over-smoothing have been established. Keriven [32] provided a comprehensive theoretical analysis of graph over-smoothing, establishing mathematical conditions under which GNNs lose their discriminative power. This theoretical framework offers crucial insights into the over-smoothing phenomenon and provides guidance for designing more robust architectures.

Building on both theoretical insights and practical considerations, several modern decoupling approaches have emerged to address over-smoothing. Shen et al. [33] proposed a block-based adaptive decoupling framework that separates different types of information processing within GNNs. Their approach uses learnable coefficients to balance GNN and MLP components, demonstrating the effectiveness of adaptive information flow control. More recently, Shen et al. [34] introduced a dual-dimensional decoupling method that resists over-smoothing by separating feature transformation and propagation along two orthogonal dimensions. This approach shows that careful separation of different information processing pathways can effectively maintain node distinguishability in deeper networks.

Recent work has also focused on universal strategies for addressing over-smoothing. Cheng et al. [35] developed SD-GCN, a universal Smoothing Deceleration strategy that reduces the smoothing speed rate through Class-Related Smoothing Deceleration (CR-SD) loss and Neighborhood Adaptive Residual (NAR). Their approach acknowledges the dual nature of smoothing, preserving beneficial intra-class smoothing while reducing harmful inter-class smoothing. Complementarily, Zhou et al. [36] introduced PSNR-GNN, a Posterior-Sampling-based Node-Adaptive Residual module that learns posterior distributions of residual coefficients for each node. This method achieves fine-grained node-level adaptivity by sampling from learned distributions, enabling more effective utilization of multi-order neighborhood information while avoiding the drawbacks of fixed coefficient methods.

Attention mechanisms have also been explored as a means of combating over-smoothing. Veličković et al. [14] proposed Graph Attention Networks (GAT), which use attention to selectively aggregate information from neighboring nodes, potentially reducing the impact of over-smoothing. However, as pointed out by Song et al. [9], attention mechanisms alone may not fully solve the over-smoothing problem in very deep networks, indicating the need for more sophisticated solutions.

Song et al. [9] introduced an Ordered GNN that addresses both heterophily and over-smoothing through a gating mechanism. This mechanism controls the flow of information across layers, allowing the model to selectively propagate relevant information. The approach has demonstrated strong effectiveness in mitigating over-smoothing while maintaining performance on heterophilic graphs.

Although this paper focuses on a gating mechanism approach, it is worth noting that other architectures have also made significant progress in addressing heterophily, particularly by leveraging graph structure. Models like MixHop [20] and FSGNN [37] harness the power of the transition matrix, allowing them to capture long-range dependencies and effectively manage heterophilic graphs. Furthermore, approaches such as Similarity-navigated GNN [38] combine both graph structure and node features, offering a balanced

approach that performs well in heterophilic settings. While these methods fall outside the scope of this work, they represent valuable contributions to the field.

Our work builds upon the advancements made in previous research, particularly those related to gating mechanisms and adaptive decoupling approaches. However, our method differs significantly from existing approaches in several key aspects. Unlike the block-based decoupling framework of Shen et al. [33], which focuses on separating information at the block level, our approach employs a three-pathway integration (GNN, MLP-current, MLP-original) with two adaptive gating coefficients (α and β) that dynamically balance information flow based on both local and global graph properties. While the dual-dimensional decoupling method [34] separates transformation and propagation operations, our method goes further by incorporating graph-aware operator selection that automatically switches between GCN and SAGE operators based on graph homophily characteristics. Additionally, our β -gated pathway for preserving original node features is specifically designed to address the challenges posed by heterophilic settings, where maintaining access to initial node characteristics is crucial for effective classification.

We propose a novel adaptive gating strategy that effectively addresses both heterophily and over-smoothing, making GNNs more suitable for a wide range of graph structures and node relationships. By tackling these challenges in a unified framework with strong theoretical motivation from recent advances [32] and building upon successful decoupling strategies [33,34], our approach pushes the boundaries of what GNNs can achieve, extending their applicability to more complex and diverse graph-structured problems.

3. Preliminaries

3.1. Notations

Let $\mathcal{G} = (\mathcal{V}, \mathcal{E})$ denote an undirected graph, where $\mathcal{V}$ represents the set of nodes (or vertices) with $|\mathcal{V}|$ denoting its cardinality, and $\mathcal{E} \subseteq \mathcal{V} \times \mathcal{V}$ represents the set of edges connecting pairs of nodes. Throughout this paper, we use $n = |\mathcal{V}|$ to denote the number of nodes and $m = |\mathcal{E}|$ to denote the number of edges in the graph.

The graph structure can be represented by its adjacency matrix $\mathbf{A} \in \{0, 1\}^{n \times n}$, where $A_{ij} = 1$ indicates the presence of an edge between nodes v_i and v_j , and $A_{ij} = 0$ indicates the absence of such an edge. For a weighted graph, $A_{ij} \in \mathbb{R}$ represents the weight of the edge between nodes v_i and v_j . The degree matrix $\mathbf{D} \in \mathbb{R}^{n \times n}$ is a diagonal matrix where $D_{ii} = \sum_{j=1}^n A_{ij}$ represents the degree of node v_i (the sum of weights of all edges connected to node v_i).

We denote several types of neighborhoods for a node v_i :

- $\mathcal{N}_i = \{v_j : (i, j) \in \mathcal{E}\}$ is the set of all nodes directly connected to node v_i (1-hop neighbors).
- $\mathcal{N}_i^k = \{v_j : \text{shortest path from } v_i \text{ to } v_j \text{ has length } k\}$ is the set of all nodes exactly k hops away from node v_i (k -hop neighbors).
- $\tilde{\mathcal{N}}_i = \mathcal{N}_i \setminus \{v_i\}$ is the neighborhood of node v_i excluding the node itself (neighbors without self-loop).

Node features are represented by matrix $\mathbf{X} \in \mathbb{R}^{n \times d_0}$, where d_0 is the input feature dimension. The feature vector of node v_i is denoted as $\mathbf{x}_i \in \mathbb{R}^{d_0}$, which corresponds to the i -th row of $\mathbf{X}$.

For labeled nodes, $y_i \in \{1, 2, \dots, C\}$ denotes the class label of node v_i , where C is the total number of classes. The one-hot encoded representation of y_i is denoted as $\mathbf{y}_i \in \{0, 1\}^C$, where $y_{ic} = 1$ if node v_i belongs to class c , and $y_{ic} = 0$ otherwise.

In the context of Graph Neural Networks, we use $\mathbf{H}^{(l)} \in \mathbb{R}^{n \times d_l}$ to denote the node representations at layer l , where d_l is the dimension of the feature space at that layer. The representation of node v_i at layer l is denoted as $\mathbf{h}_i^{(l)} \in \mathbb{R}^{d_l}$, which corresponds to the i -th row of $\mathbf{H}^{(l)}$. The initial node representations are set as $\mathbf{H}^{(0)} = \mathbf{X}$.

For graph modification operations, we use the tilde notation to indicate the self-looped version of a graph component: $\tilde{\mathbf{A}} = \mathbf{A} + \mathbf{I}$ is the adjacency matrix with self-loops added, and $\tilde{\mathbf{D}}$ is the corresponding degree matrix with $\tilde{D}_{ii} = \sum_{j=1}^n \tilde{A}_{ij} = D_{ii} + 1$.

3.2. Graph neural networks

Graph Neural Networks learn node representations by iteratively aggregating and transforming information from the local neighborhood of each node [2]. This process enables GNNs to capture both node features and graph structural information. The key insight behind GNNs is that nodes with similar structural roles and features should have similar representations [17].

For a GNN with L layers, we denote the node representations at layer l as $\mathbf{H}^{(l)} \in \mathbb{R}^{|\mathcal{V}| \times d_l}$, where d_l is the dimension of the feature space at layer l , and $\mathbf{h}_i^{(l)}$ represents the embedding of node v_i at this layer. The initial node representations are set as $\mathbf{H}^{(0)} = \mathbf{X}$.

The message passing process in GNNs can be decomposed into two fundamental operations that are performed at each layer:

1) **Combine**: This operation aggregates information from neighboring nodes to create a message that captures the local graph structure. The combine step ensures that the model learns from the node's neighborhood while maintaining permutation invariance:

$$\mathbf{m}_i^{(l)} = \text{COMBINE}^{(l)}(\{\mathbf{h}_j^{(l-1)} : v_j \in \mathcal{N}_i\}) \quad (1)$$

2) **Update**: This operation transforms the node's current representation using both its previous state and the combined neighborhood information. The update function is crucial for maintaining the node's own features while incorporating neighborhood information:

$$\mathbf{h}_i^{(l)} = \text{UPDATE}^{(l)}(\mathbf{h}_i^{(l-1)}, \mathbf{m}_i^{(l)}) \quad (2)$$

The $\text{COMBINE}^{(l)}$ function implements permutation-invariant operations to ensure the model's output remains independent of the order in which neighbors are processed. Common aggregation schemes include taking the mean of neighbor representations [2,39],

which provides a normalized summary of the neighborhood, summing all neighbor representations to capture the total neighborhood influence, or selecting the element-wise maximum values across neighbor representations to identify the most salient features.

The $\text{UPDATE}^{(l)}$ function is typically implemented as a learnable transformation, often using neural networks. This function is critical for the model's expressiveness as it determines how the node's representation evolves through the layers. The design of this function significantly impacts the model's ability to handle the over-smoothing problem [40], especially in deeper architectures [7, 17, 26].

For node classification tasks, the final node representations $\mathbf{H}^{(L)}$ are passed through a softmax classifier, with the cross-entropy loss $\mathcal{L}$ defined as:

$$\mathcal{L} = -\frac{1}{|\mathcal{V}|} \sum_{i=1}^{|\mathcal{V}|} \sum_{c=1}^C y_{ic} \log(\text{softmax}(\mathbf{h}_i^{(L)})_c) \quad (3)$$

where C is the number of classes and y_{ic} is the ground truth label for node v_i in class c .

4. Methodology

4.1. Problem formulation

In graph-structured data, nodes often exhibit different patterns of connectivity in relation to their class labels [4]. We quantify this structural property through the edge homophily ratio h_{edge} :

$$h_{edge} = \frac{|(u, v) : (u, v) \in \mathcal{E} \wedge y_u = y_v|}{|\mathcal{E}|} \quad (4)$$

where u and v are nodes in the graph, and y_u and y_v are their respective class labels. This ratio quantifies the proportion of edges connecting nodes of the same class, with values closer to 1 indicating higher homophily (similar nodes connect) and values closer to 0 indicating higher heterophily (dissimilar nodes connect). This property significantly impacts the effectiveness of graph neural networks [41].

Real-world graphs exhibit a wide spectrum of homophily patterns, from strongly homophilic networks where connected nodes typically share the same class (e.g., social networks, citation networks) to strongly heterophilic networks where connected nodes often belong to different classes (e.g., biological networks, web graphs). Traditional GNNs, optimized for homophilic graphs, often fail in heterophilic settings due to their assumption that connected nodes should share similar features and labels [7].

This fundamental limitation poses several critical challenges for graph neural networks:

1. **Homophily assumption limitation:** Most existing GNN architectures rely heavily on the homophily assumption, leading to poor performance when this assumption is violated in heterophilic graphs.
2. **Over-smoothing in deep architectures:** As network depth increases, repeated message passing causes node representations to become overly similar, which is particularly problematic in heterophilic settings where maintaining node diversity is crucial for effective classification.
3. **Lack of adaptive mechanisms:** Current approaches typically excel in either homophilic or heterophilic settings but lack the ability to automatically adapt their information processing strategy based on the underlying graph structure.

Our goal is to develop a unified GNN architecture that can robustly handle graphs across the entire spectrum of homophily patterns by adaptively balancing structural information and node features based on the inherent characteristics of each graph.

4.2. Adaptive gating architecture

To address these challenges, we propose AG-GNN, a novel architecture that processes information through multiple specialized pathways, as illustrated in Fig. 2. Each pathway is designed to capture different aspects of the node and graph structure, enabling effective learning in both homophilic and heterophilic settings.

4.2.1. Graph component

At the core of our architecture is a flexible message passing component that adapts to the graph's homophily level. For each node v at layer l , we compute:

$$\mathbf{H}_{GNN}^{(l)} = \text{GNN}(\mathbf{H}^{(l-1)}, \mathbf{A}) \quad (5)$$

We specifically employ different GNN operators depending on the heterophily of the graph:

$$\mathbf{H}_{GNN}^{(l)} = \begin{cases} \text{GCN}(\mathbf{H}^{(l-1)}, \mathbf{A}) & \text{if homophilic graph} \\ \text{SAGE}(\mathbf{H}^{(l-1)}, \mathbf{A}) & \text{if heterophilic graph} \end{cases} \quad (6)$$

For homophilic graphs, we utilize a GCN [2] operator that normalizes neighborhood influence:

$$\text{GCN}(\mathbf{H}, \mathbf{A}) = \tilde{\mathbf{D}}^{-\frac{1}{2}} \tilde{\mathbf{A}} \tilde{\mathbf{D}}^{-\frac{1}{2}} \mathbf{H} \mathbf{W} \quad (7)$$

where $\tilde{\mathbf{A}} = \mathbf{A} + \mathbf{I}$ is the adjacency matrix with self-loops and $\tilde{\mathbf{D}}_{ii} = \sum_j \tilde{\mathbf{A}}_{ij}$.

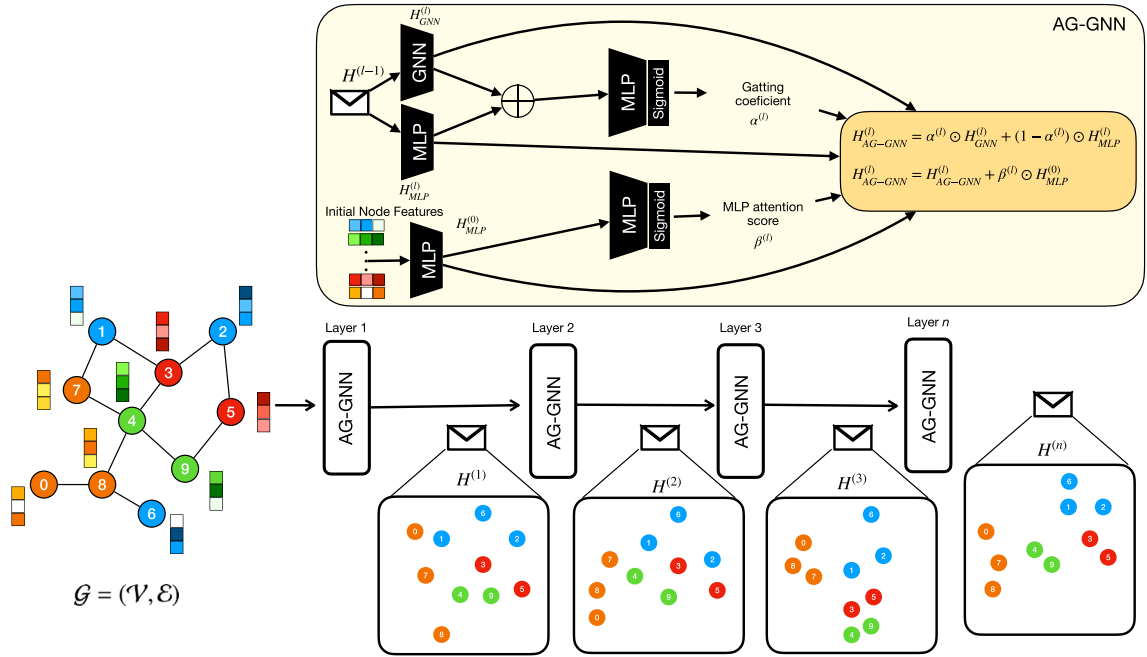


Fig. 2. Overview of AG-GNN architecture. The model processes input through multiple pathways: a GNN component captures structural information ($H_{GNN}^{(l)}$), while parallel MLP pathways process node features ($H_{MLP}^{(l)}$) and preserve initial features ($H_{MLP}^{(0)}$). These components are dynamically combined through a gating mechanism with learnable coefficients $\alpha^{(l)}$ and $\beta^{(l)}$, which adapt based on both local node properties and global graph structure. The bottom part illustrates how node embeddings evolve through successive AG-GNN layers, preserving meaningful structural relationships while avoiding over-smoothing. Node colors represent different classes, and their spatial arrangement in the embedding space demonstrates the model's ability to maintain class separability.

For heterophilic graphs, we employ a SAGE [39] operator that maintains explicit separation between ego and neighbor features:

$$\text{SAGE}(\mathbf{h}_v, \mathbf{h}_u : u \in \mathcal{N}(v)) = \mathbf{W}_1 \mathbf{h}_v + \mathbf{W}_2 \cdot \text{MEAN}(\mathbf{h}_u : u \in \mathcal{N}(v)) \quad (8)$$

In homophilic settings (e.g., $h_{edge} \approx 0.8$), the GNN component effectively aggregates similar information from neighboring nodes of the same class, enhancing class-specific features. For instance, in citation networks where papers often cite others from the same field, this pathway strengthens the thematic signal by combining similar node representations.

In contrast, for heterophilic settings (e.g., $h_{edge} \approx 0.2$), as in protein interaction networks where different protein types tend to interact, the SAGE operator's ego-neighbor separation prevents the dilution of distinctive node features while still leveraging structural information.

4.2.2. Feature processing pathways

Our architecture processes node features through two complementary MLP pathways. The first pathway applies a non-linear transformation to the representation from the previous layer, focusing on global feature characteristics rather than neighborhood aggregation:

$$\begin{aligned} \mathbf{H}_{MLP}^{(l)} &= \text{LayerNorm}(\mathbf{W}_1 \mathbf{H}^{(l-1)} + b_1) \\ \mathbf{H}_{MLP}^{(l)} &= \mathbf{W}_2 \cdot \text{ReLU}(\mathbf{H}_{MLP}^{(l)}) + b_2 \end{aligned} \quad (9)$$

The second pathway maintains a direct connection to the original features through a similar transformation:

$$\begin{aligned} \mathbf{H}_{MLP}^{(0)} &= \text{LayerNorm}(\tilde{\mathbf{W}}_1 \mathbf{X} + \tilde{b}_1) \\ \mathbf{H}_{MLP}^{(0)} &= \tilde{\mathbf{W}}_2 \cdot \text{ReLU}(\mathbf{H}_{MLP}^{(0)}) + \tilde{b}_2 \end{aligned} \quad (10)$$

This second pathway is particularly valuable for heterophilic graphs, where most neighbors belong to different classes. By maintaining access to original features, the model can preserve distinctive node characteristics that might otherwise be obscured through repeated neighborhood aggregation.

In protein interaction networks, for example, this pathway preserves specific protein features that might be crucial for classification, even as structural information is incorporated through other pathways.

4.3. Adaptive integration mechanism

4.3.1. Gating coefficient computation and properties

Our model's key innovation is its sophisticated gating mechanism that dynamically balances information from the graph and MLP components. The two primary gating coefficients, $\alpha^{(l)}$ and $\beta^{(l)}$, are computed as follows:

For $\alpha^{(l)}$, which determines the balance between GNN and MLP representations:

$$\begin{aligned}\alpha^{(l)} &= \text{LayerNorm}(W_\alpha^1 [\mathbf{H}_{GNN}^{(l)} | \mathbf{H}_{MLP}^{(l)}] + b_\alpha^1) \\ \alpha^{(l)} &= \sigma(W_\alpha^2 \cdot \text{ReLU}(\alpha^{(l)}) + b_\alpha^2)\end{aligned}\tag{11}$$

where $W_\alpha^2 \in \mathbb{R}^{d_h \times 1}$ projects the transformed representations to a scalar importance weight and σ is the sigmoid function, ensuring that $\alpha^{(l)} \in [0, 1]$.

For $\beta^{(l)}$, which determines the contribution of the initial features:

$$\beta^{(l)} = \sigma(W_\beta \mathbf{H}_{MLP}^{(0)})\tag{12}$$

where $W_\beta \in \mathbb{R}^{d_h \times 1}$ projects the transformed initial features to a scalar importance weight.

4.3.2. Initialization and training constraints

The gating coefficients $\alpha^{(l)}$ and $\beta^{(l)}$ are initialized with small random values drawn from a uniform distribution before being transformed by the sigmoid function. This ensures that initially, the model assigns approximately equal importance to all information pathways, allowing the training process to discover the optimal balance.

During training, these coefficients are constrained to the range $[0, 1]$ through the sigmoid function, which ensures numerical stability and interpretability. The weights W_α^1 , W_α^2 , and W_β are updated through backpropagation, allowing the model to learn the optimal information flow pattern for each specific graph.

4.3.3. Final integration

The adaptive integration of the three representation pathways is summarized as:

$$\mathbf{H}^{(l)} = \alpha^{(l)} \odot \mathbf{H}_{GNN}^{(l)} + (1 - \alpha^{(l)}) \odot \mathbf{H}_{MLP}^{(l)} + \beta^{(l)} \odot \mathbf{H}_{MLP}^{(0)}\tag{13}$$

where $\odot$ is the Hadamard product.

4.3.4. Pathway contributions in different scenarios

Our experimental analysis reveals distinct patterns in how $\alpha^{(l)}$ and $\beta^{(l)}$ evolve during training:

- **In homophilic settings** (e.g., citation networks with $h_{edge} \approx 0.8$), $\alpha^{(l)}$ typically converges to higher values (0.6–0.8), indicating a stronger reliance on the GNN component. The $\beta^{(l)}$ values tend to decrease in deeper layers, as neighborhood information becomes more informative than initial features. For instance, in the Cora dataset, we observe $\alpha^{(4)} \approx 0.75$ and $\beta^{(4)} \approx 0.2$ in the fourth layer.
- **In heterophilic settings** (e.g., web networks with $h_{edge} \approx 0.2$), $\alpha^{(l)}$ typically converges to lower values (0.3–0.5), indicating a stronger reliance on the MLP pathway. The $\beta^{(l)}$ values remain higher even in deeper layers, reflecting the continued importance of initial features. For example, in the Chameleon dataset, we observe $\alpha^{(4)} \approx 0.35$ and $\beta^{(4)} \approx 0.4$ in the fourth layer.

This adaptive behavior illustrates how our model learns to allocate computational resources differently based on the graph structure's inherent properties, automatically switching between structure-focused learning (when homophily is high) and feature-focused learning (when heterophily is high).

4.4. Training and optimization

The model parameters are optimized end-to-end using cross-entropy loss computed over the labeled nodes. We can summarize the training procedure in [Algorithm 1](#):

4.5. Complexity analysis

The computational complexity of AG-GNN warrants careful analysis across its main operations. The neighborhood aggregation step requires $\mathcal{O}(|\mathcal{E}|d)$ operations to process all edges in the graph, where d is the feature dimension. The feature processing pathway incurs in $\mathcal{O}(|\mathcal{V}|d^2)$ operations due to the dense linear transformations applied to each node's features. The gating mechanism requires an additional $\mathcal{O}(|\mathcal{V}|d^2)$ operations for computing the attention weights. The total computational complexity per layer thus becomes $\mathcal{O}(|\mathcal{E}|d + |\mathcal{V}|d^2)$. For a model with L layers, the final complexity is $\mathcal{O}(L(|\mathcal{E}|d + |\mathcal{V}|d^2))$. While this is comparable to standard GNN architectures, our model achieves superior expressiveness through its adaptive mechanisms without introducing significant computational overhead. The parallel nature of our computations allows for efficient implementation on modern hardware accelerators.

Algorithm 1: Adaptive graph neural network training.

Input: Graph $\mathcal{G} = (\mathcal{V}, \mathcal{E})$, Features $\mathbf{X}$, Labels $\mathbf{Y}$, Layers L
Output: Optimized model parameters

```

1 Initialize network parameters;  $\mathbf{H}^{(0)} \leftarrow \mathbf{X}$ ;
2 for each epoch do
3   for  $l = 1$  to  $L$  do
4     // Graph Transformations
5      $\mathbf{H}_{GNN}^{(l)} \leftarrow \text{Eq. (5)}$ ;
6     // MLP Transformations
7      $\mathbf{H}_{MLP}^{(l)} \leftarrow \text{Eq. (9)}$ ;
8     // Initial Features Transformations
9      $\mathbf{H}_{MLP}^{(0)} \leftarrow \text{Eq. (10)}$ ;
10    // Compute Gates
11     $\alpha^{(l)} \leftarrow \text{Eq. (11)}$ ;
12     $\beta^{(l)} \leftarrow \text{Eq. (12)}$ ;
13    // Update Node Representation
14     $\mathbf{H}^{(l)} \leftarrow \alpha^{(l)} \odot \mathbf{H}_{GNN}^{(l)} + (1 - \alpha^{(l)}) \odot \mathbf{H}_{MLP}^{(l)} + \beta^{(l)} \odot \mathbf{H}_{MLP}^{(0)}$ ;
15  end
16  Compute loss  $\mathcal{L}$  according to Eq. (3);
17  Update parameters via Adam optimizer;
18 end
19 return Optimized parameters

```

5. Experiments

5.1. Experimental settings

We conduct comprehensive experiments on 15 diverse benchmark datasets to evaluate our proposed model. The experimental evaluation consists of nine small-scale networks that are commonly used in the literature [4] (Texas, Wisconsin, Actor, Squirrel, Chameleon, Cornell, Citeseer, Pubmed, and Cora) [4,42,43] and six large-scale real-world graphs [18,23] (Penn94, Pokec, ArXiv-year, Snap-patents, Genius, and Twitch-gamers). These datasets represent a wide spectrum of network properties, with homophily ratios ranging from 0.07 to 0.81, sizes varying from 183 to 2,923,922 nodes, and diverse structural characteristics. For small-scale networks, we follow the standard setup used in previous works, employing 10 random splits with 48 %, 32 %, and 20 % of nodes per class for training, validation, and testing, respectively [4,7,17]. For large-scale networks, we maintain consistency with the original benchmark splits (50 %, 25 %, 25 %) to ensure fair comparison [18]. The above configurations were extracted from Table 5 [7] and Table 3 in [18]. All experiments are implemented using PyTorch and PyTorch Geometric frameworks, and all the hyperparameter settings and the best configurations can be found in our GitHub repository.¹

5.2. Performance analysis

The experimental results are presented in Tables 1–3, demonstrating the effectiveness of our proposed AG-GNN model across different network scales and homophily levels.

5.2.1. Small-scale networks

On small-scale networks, AG-GNN demonstrates remarkable performance across the entire spectrum of homophily levels. In strongly heterophilic networks ($h \leq 0.3$), our model achieves state-of-the-art results with significant margins. Particularly noteworthy is its performance in Texas and Wisconsin, where it achieves accuracies of 86.49 % and 88.03 % respectively, surpassing the previous best results by 1.63 % and 1.16 %. The improvement is even more pronounced on Cornell, where AG-GNN reaches 87.02 % accuracy, establishing a new state-of-the-art with a 2.16 % margin over the next best method.

For networks with moderate homophily ($0.3 < h < 0.7$), AG-GNN maintains its superior performance. On the challenging Actor dataset, it achieves 37.68 % accuracy. The improvement is particularly significant on Chameleon, where our model reaches 70.02 % accuracy, demonstrating its ability to handle complex network structures effectively.

In homophilic networks ($h \geq 0.7$), AG-GNN continues to show strong performance. On Citeseer, Pubmed, and Cora, it achieves accuracies of 77.46 %, 89.60 %, and 88.10 % respectively. This strong performance in homophilic settings can be attributed to our adaptive gating mechanism's ability to effectively emphasize the GNN pathway when neighborhood aggregation is beneficial. Analysis of the learned α values reveals that in these homophilic datasets, the model assigns higher importance (typically

¹ <https://github.com/AhmedBeggaUA/InformationScience>

Table 1
Small real-world datasets. Top three models are colored by **First**, **Second**, and **Third**.

	Texas	Wisconsin	Actor	Squirrel	Chameleon	Cornell	Citeseer	Pubmed	Cora	
Hom. level h	0.11	0.21	0.22	0.22	0.23	0.3	0.74	0.8	0.81	Avg
#Nodes	183	251	7,600	5,201	2,277	183	3,327	19,717	2,708	Rank
#Edges	295	466	26,752	198,493	31,421	280	4,676	44,327	5,278	
#Classes	5	5	5	5	5	5	7	3	6	
AG-GNN	86.49±6.72	88.03±4.33	37.68±0.96	54.72±1.10	70.02±2.02	87.02±2.02	77.46±1.53	89.60±0.26	88.10±1.05	1.89
GGCN	84.86±4.55	86.86±3.29	37.54±1.56	55.17±1.58	71.14±1.84	85.68±6.63	77.14±1.45	89.15±0.37	87.95±1.05	2.89
PSNR-GCN	81.20±5.80	84.50±3.80	36.20±1.10	52.80±1.70	67.30±2.10	84.20±4.50	76.90±1.30	88.70±0.40	87.20±1.20	5.67
SD-GCN	79.03±6.80	82.21±4.12	35.88±1.37	54.69±2.01	69.41±2.00	83.50±5.20	76.80±1.40	88.90±0.35	87.50±1.15	5.78
H ₂ GCN	84.86±7.23	87.65±4.98	37.44±1.30	36.48±1.86	69.41±2.15	82.70±5.28	77.11±1.57	90.15±0.38	87.87±1.20	4.33
GCNII	77.57±3.83	80.39±3.4	36.53±1.30	38.47±1.58	63.86±3.04	77.86±3.79	77.33±1.48	89.95±0.43	88.37±1.25	6.67
GCN+AEE+BDE	78.50±6.20	81.80±4.60	34.90±1.25	51.20±1.80	65.80±2.30	82.10±5.80	76.50±1.45	88.20±0.45	86.80±1.30	8.22
GPRGNN	78.38±4.36	82.94±4.21	34.63±1.22	31.61±1.24	46.58±1.71	80.27±8.11	78.02±1.67	87.54±0.38	87.95±1.18	8.22
Geom-GCN	66.76±2.72	64.51±3.66	31.59±1.15	38.15±0.92	60.00±2.81	60.54±3.67	78.02±1.15	89.95±0.47	85.35±1.57	9.33
PairNorm	60.27±4.34	48.43±6.14	27.40±1.24	50.44±2.04	62.74±2.82	58.92±3.15	73.59±1.47	87.53±0.44	85.79±1.01	11.88
GraphSAGE	82.43±6.14	81.18±5.56	34.23±0.99	41.61±0.74	58.73±1.68	75.95±5.01	76.04±1.30	88.45±0.50	86.90±1.04	9.22
GAT	52.16±6.63	49.41±4.09	27.44±0.89	40.72±1.55	60.26±2.5	61.89±5.05	76.55±1.23	86.33±0.48	87.30±1.10	11.00
GCN	55.14±5.16	51.76±3.06	27.32±1.10	53.43±2.01	64.82±2.24	60.54±5.3	76.50±1.36	88.42±0.57	86.98±1.27	10.00
MLP	80.81±4.75	85.29±3.31	36.53±0.70	28.77±1.56	46.21±2.99	81.89±6.40	74.02±1.90	87.16±0.37	75.69±2.00	9.89

Table 2
Large real-world datasets. Top three models are colored by **First**, **Second**, and **Third**.

	Penn94	pokec	arXiv-year	snap-patents	genius	twitch-gamers	
Edge Hom.	0.47	0.44	0.22	0.07	0.61	0.54	Avg
#Nodes	41,554	1,632,803	169,343	2,923,922	421,961	168,114	Rank
#Edges	1,362,229	30,622,564	1,166,243	13,975,788	984,979	6,797,557	
#Features	5	65	128	269	12	7	
#Classes	2	2	5	5	2	2	
AG-GNN	88.78±0.87	79.16±0.21	51.36±0.83	46.27±0.28	88.84±0.34	64.05±0.97	1.50
GCNII	82.92±0.59	78.94±0.11	47.21±0.28	37.88±0.69	90.24±0.09	63.39±0.61	3.17
SD-GCN	82.10±0.45	77.50±0.18	48.20±0.55	44.80±0.25	87.20±0.28	62.50±0.85	3.33
PSNR-GCN	81.80±0.52	76.90±0.22	47.80±0.48	43.90±0.30	88.50±0.32	61.80±0.75	4.83
GGCN	OOM	OOM	OOM	OOM	OOM	OOM	-
H ₂ GCN	81.31±0.60	OOM	49.09±0.10	OOM	OOM	OOM	-
GPRGNN	81.38±0.16	78.83±0.05	45.07±0.21	40.19±0.03	90.05±0.31	61.89±0.29	5.33
GCN+AEE+BDE	80.50±0.65	75.20±0.28	46.50±0.62	42.60±0.35	87.80±0.38	60.90±0.95	6.67
GAT	81.53±0.55	71.77±6.18	46.05±0.51	45.37±0.44	55.80±0.87	59.89±4.12	6.83
GCN	82.47±0.27	75.45±0.17	46.02±0.26	45.65±0.04	87.42±0.37	62.18±0.26	4.83
MLP	73.61±0.40	62.37±0.02	36.70±0.21	31.34±0.05	86.68±0.09	60.92±0.07	8.50

0.65–0.75) to the GNN component while reducing reliance on initial features as network depth increases. This demonstrates that our model automatically detects the homophilic nature of these graphs and adjusts its information flow accordingly. Unlike specialized heterophilic methods that may sacrifice performance on homophilic graphs, our approach maintains competitive accuracy across the entire homophily spectrum, showing less than 1 % difference from specialized homophilic methods like GCNII on these datasets.

5.2.2. Large-scale networks

The performance of AG-GNN on large-scale networks, presented in Table 2, further validates its effectiveness and scalability. Most notably, several baseline models, including GGCN and H2GCN, fail to scale to these larger datasets (indicated by OOM - Out-of-Memory), while AG-GNN maintains both computational efficiency and superior performance. On strongly heterophilic graphs, AG-GNN demonstrates exceptional performance. For ArXiv-year ($h = 0.22$), it achieves 51.36 % accuracy, surpassing the previous best result by 2.27 %. Even more impressively, on Snap-patents ($h = 0.07$), AG-GNN reaches 46.27 % accuracy, outperforming all scalable baselines by a significant margin of 0.62 %. These results are particularly significant given the extreme heterophily and scale of these networks. For networks with moderate homophily, AG-GNN continues to excel. On Penn94 ($h = 0.47$), it achieves 88.78 % accuracy, surpassing the next-best method by 5.86 %. Similarly, on Pokec ($h = 0.44$), AG-GNN reaches 79.16 % accuracy, demonstrating a 0.33 % improvement over the closest competitor. The consistent performance across these diverse datasets underscores the model's robustness and adaptability. Note that for binary classes, we compute the AUC as they do in [44].

5.3. Model depth analysis

Table 3 presents a detailed analysis of model performance across different network depths, ranging from 2 to 64 layers. This comprehensive study reveals several crucial insights about the model's behavior and stability. We evaluated our model following the configurations from Table 2 in [17].

Table 3

Model performance for different depths. We report the accuracy, the $\pm$ stdev over different data splits and the layer at which the best performance is achieved. 'OOM': Out-Of-Memory; 'INS': numerical instability.

Layers	2	4	8	16	32	64	Best	2	4	8	16	32	64	Best
Cora ($h=0.81$)								Citeseer ($h=0.74$)						
AG-GNN	87.7 $\pm$ 1.2	88.0$\pm$1.3	87.5 $\pm$ 1.2	87.5 $\pm$ 1.2	87.6 $\pm$ 1.0	87.8 $\pm$ 1.3	4	75.8 $\pm$ 1.7	77.1$\pm$1.5	76.9 $\pm$ 1.5	76.9 $\pm$ 1.3	76.9 $\pm$ 1.5	76.9 $\pm$ 1.4	4
GGCN	87.0 $\pm$ 1.2	87.5 $\pm$ 1.3	87.6 $\pm$ 1.3	87.5 $\pm$ 1.2	88.0$\pm$1.1	87.3 $\pm$ 1.4	32	76.8 $\pm$ 1.8	76.8 $\pm$ 1.5	76.9 $\pm$ 1.5	76.9 $\pm$ 1.6	77.0$\pm$1.5	76.7 $\pm$ 1.4	32
GPRGNN	87.9 $\pm$ 1.1	88.0$\pm$1.2	87.9 $\pm$ 1.4	87.3 $\pm$ 1.5	87.2 $\pm$ 1.3	87.3 $\pm$ 1.2	4	77.1$\pm$1.7	77.1 $\pm$ 1.4	77.1 $\pm$ 1.6	76.0 $\pm$ 1.6	74.5 $\pm$ 1.5	74.4 $\pm$ 1.7	2
H2GCN	87.6$\pm$1.2	86.1 $\pm$ 1.5	86.2 $\pm$ 2.1	OOM	OOM	OOM	2	76.9$\pm$1.8	76.1 $\pm$ 1.5	74.1 $\pm$ 1.8	OOM	OOM	OOM	2
SD-GCN	86.4 $\pm$ 1.2	86.8 $\pm$ 1.3	86.6 $\pm$ 1.2	87.1 $\pm$ 1.0	87.5$\pm$1.4	87.2 $\pm$ 1.3	32	76.8 $\pm$ 1.6	77.1$\pm$1.7	76.7 $\pm$ 1.6	75.8 $\pm$ 1.9	74.3 $\pm$ 2.1	74.5 $\pm$ 2.3	4
GCNII	85.4 $\pm$ 1.6	85.4 $\pm$ 1.5	86.4 $\pm$ 1.0	87.1 $\pm$ 1.1	88.0 $\pm$ 1.1	88.4$\pm$1.3	64	75.4 $\pm$ 1.8	75.3 $\pm$ 1.9	76.0 $\pm$ 1.7	77.0 $\pm$ 1.4	77.3$\pm$1.5	77.2 $\pm$ 1.5	32
PairNorm	85.8$\pm$1.0	85.1 $\pm$ 0.9	84.7 $\pm$ 1.1	82.2 $\pm$ 2.8	60.3 $\pm$ 8.3	44.4 $\pm$ 5.6	2	73.6$\pm$1.5	72.6 $\pm$ 2.0	72.3 $\pm$ 1.6	59.7 $\pm$ 16	27.2 $\pm$ 11	23.8 $\pm$ 6.6	2
Geom	85.4$\pm$1.6	21.0 $\pm$ 2.6	14.0 $\pm$ 1.5	14.0 $\pm$ 1.5	14.0 $\pm$ 1.5	14.0 $\pm$ 1.5	2	78.0$\pm$1.2	23.0 $\pm$ 2.0	7.2 $\pm$ 0.9	7.2 $\pm$ 0.9	7.2 $\pm$ 0.9	7.2 $\pm$ 0.9	2
GCN	87.0$\pm$1.3	83.2 $\pm$ 1.6	31.0 $\pm$ 3.1	31.1 $\pm$ 3.0	30.8 $\pm$ 3.5	31.9 $\pm$ 2.1	2	76.5$\pm$1.4	64.3 $\pm$ 8.3	24.2 $\pm$ 1.7	23.1 $\pm$ 3.0	25.5 $\pm$ 1.8	24.7 $\pm$ 1.7	2
GAT	87.3$\pm$1.1	86.5 $\pm$ 1.2	85.0 $\pm$ 1.2	INS	INS	INS	2	76.6$\pm$1.2	75.3 $\pm$ 1.4	66.6 $\pm$ 5.1	INS	INS	INS	2
Cornell ($h=0.3$)								Chameleon ($h=0.23$)						
AG-GNN	82.8 $\pm$ 5.3	85.7 $\pm$ 6.3	85.2 $\pm$ 6.6	84.8 $\pm$ 5.5	84.4 $\pm$ 6.1	86.1$\pm$5.4	64	65.3 $\pm$ 2.1	64.4 $\pm$ 1.9	64.5 $\pm$ 2.1	65.0 $\pm$ 1.4	63.1 $\pm$ 1.7	70.2$\pm$2.0	64
GGCN	83.8 $\pm$ 6.7	83.8 $\pm$ 6.2	84.9$\pm$5.7	83.8 $\pm$ 6.7	83.8 $\pm$ 6.5	84.3 $\pm$ 5.9	8	70.8$\pm$1.4	69.6 $\pm$ 2.7	70.3 $\pm$ 1.7	70.4 $\pm$ 1.8	70.3 $\pm$ 1.6	70.2 $\pm$ 2.0	2
GPRGNN	76.8 $\pm$ 8.2	77.6 $\pm$ 7.5	80.3$\pm$8.1	78.4 $\pm$ 6.0	74.6 $\pm$ 7.7	70.0 $\pm$ 5.7	8	46.6$\pm$1.8	45.7 $\pm$ 3.5	41.2 $\pm$ 5.8	39.6 $\pm$ 7.9	35.4 $\pm$ 8.5	36.4 $\pm$ 2.4	2
H2GCN	81.9 $\pm$ 6.0	82.7$\pm$6.3	80.3 $\pm$ 6.6	OOM	OOM	OOM	4	59.1 $\pm$ 1.9	60.1$\pm$2.2	OOM	OOM	OOM	OOM	4
SD-GCN	82.48 $\pm$ 5.2	80.8 $\pm$ 6.4	82.93$\pm$6.2	81.3 $\pm$ 5.9	80.3 $\pm$ 6.4	79.8 $\pm$ 6.2	8	62.2 $\pm$ 3.1	65.5 $\pm$ 3.5	67.4$\pm$2.8	66.6 $\pm$ 2.9	65.6 $\pm$ 3.3	65.3 $\pm$ 3.2	8
GCNII	67.6 $\pm$ 11.3	64.6 $\pm$ 9.6	73.2 $\pm$ 5.9	77.8$\pm$4.0	75.4 $\pm$ 5.5	73.8 $\pm$ 4.4	16	61.1 $\pm$ 4.1	63.9$\pm$3.0	62.9 $\pm$ 1.8	60.2 $\pm$ 2.1	57.0 $\pm$ 1.8	55.9 $\pm$ 2.3	4
PairNorm	50.3 $\pm$ 1.8	53.5 $\pm$ 8.0	58.4 $\pm$ 5.0	58.4 $\pm$ 4.9	58.9$\pm$3.2	58.9 $\pm$ 3.2	32	62.7$\pm$2.8	59.0 $\pm$ 2.8	54.1 $\pm$ 2.2	46.4 $\pm$ 3.2	46.8 $\pm$ 2.3	46.3 $\pm$ 3.2	2
Geom	60.5$\pm$3.7	23.8 $\pm$ 11.6	13.0 $\pm$ 2.9	13.0 $\pm$ 2.9	13.0 $\pm$ 2.9	13.0 $\pm$ 2.9	2	60.0$\pm$2.8	19.2 $\pm$ 1.7	19.6 $\pm$ 1.7	19.6 $\pm$ 1.7	19.6 $\pm$ 1.7	19.6 $\pm$ 1.7	2
GCN	60.5$\pm$5.3	59.2 $\pm$ 3.3	58.9 $\pm$ 3.2	58.9 $\pm$ 3.2	58.9 $\pm$ 3.2	58.9 $\pm$ 3.2	2	64.8$\pm$2.2	53.1 $\pm$ 4.4	35.2 $\pm$ 3.1	35.4 $\pm$ 3.2	35.2 $\pm$ 3.3	35.5 $\pm$ 3.1	2
GAT	61.9$\pm$5.1	58.4 $\pm$ 4.1	58.4 $\pm$ 3.9	INS	INS	INS	2	60.3$\pm$2.5	48.7 $\pm$ 3.0	35.1 $\pm$ 3.6	INS	INS	INS	2

Table 4

Ablation study of AG-GNN components. We report classification accuracy (%) on three datasets with different homophily levels. AG-GNN (full) represents our complete model; AG-GNN (w/o β) removes the initial feature retention pathway by setting $\beta = 0$; AG-GNN (w/o α) removes the adaptive gating between GNN and MLP paths by setting $\alpha = 0.5$.

Model	Texas ($h=0.11$)	Cornell ($h=0.3$)	Cora ($h=0.81$)
AG-GNN (full)	86.49$\pm$6.72	87.02$\pm$2.02	88.10$\pm$1.05
AG-GNN (w/o β)	78.11 $\pm$ 5.93	81.35 $\pm$ 3.27	87.85 $\pm$ 1.23
AG-GNN (w/o α)	80.27 $\pm$ 6.32	83.51 $\pm$ 2.88	86.72 $\pm$ 1.31

In homophilic settings, AG-GNN demonstrates remarkable stability and performance. On Cora ($h = 0.81$), the model achieves optimal performance of 88.0 % at 4 layers and maintains consistent performance up to 64 layers, with variations less than 0.5 %. This stability is particularly noteworthy when compared to baseline models such as GCN and GAT, which experience severe performance degradation beyond 4 layers, dropping to accuracies below 32 % and becoming unstable (INS) respectively. Similarly, on Citeseer ($h = 0.74$), AG-GNN reaches its peak performance of 77.1 % at 4 layers and shows minimal degradation as depth increases, maintaining performance around 76.9 % even at 64 layers. In contrast, baseline models exhibit significant performance collapse, with PairNorm dropping to 23.8 % and Geom-GCN falling to 7.2 % at deeper layers.

The depth analysis on heterophilic networks reveals even more interesting patterns. On Cornell ($h = 0.3$), AG-GNN uniquely benefits from increased depth, achieving its best performance of 86.1 % at 64 layers. This behavior contrasts sharply with baseline models, which either fail to run at greater depths (H2GCN) or show significant performance degradation (GPRGNN drops from 80.3 % to 70.0 %). The results on Chameleon ($h = 0.23$) further emphasize AG-GNN's capability to leverage deeper architectures effectively. The model achieves 70.2 % accuracy at 64 layers, significantly outperforming all baselines. This ability to maintain and even improve performance at greater depths in heterophilic settings demonstrates the effectiveness of our adaptive gating mechanism in preventing over-smoothing while capturing complex structural patterns.

5.4. Ablation study

To validate the contribution of individual components in our AG-GNN architecture, particularly the role of the gating coefficients $\alpha^{(l)}$ and $\beta^{(l)}$, we conduct a comprehensive ablation study. Table 4 presents the results on three representative datasets with varying homophily levels: Texas (highly heterophilic, $h = 0.11$), Cornell (moderately heterophilic, $h = 0.3$), and Cora (homophilic, $h = 0.81$).

The results clearly demonstrate the importance of both adaptive components, particularly in heterophilic settings:

1. **Impact of the β term:** Removing the β term, which controls the contribution of original features, leads to an 8.38 % accuracy decrease on Texas and a 5.67 % decrease on Cornell. This confirms our hypothesis that preserving original features is crucial in

heterophilic settings where neighborhood aggregation might introduce misleading information. Even in the homophilic Cora dataset, we still observe a slight decrease of 0.25 %, though the impact is substantially smaller.

2. **Impact of the adaptive α coefficient:** Setting α to a fixed value of 0.5 (removing its adaptive nature) degrades performance across all datasets, with reductions of 6.22 % on Texas, 3.51 % on Cornell, and 1.38 % on Cora. This demonstrates the importance of adaptively balancing the GNN and MLP pathways based on both local node properties and global graph structure.

These findings support our design decisions and demonstrate that the parallel pathways with adaptive integration are essential for the model's effectiveness, particularly in heterophilic settings. The larger performance gaps in heterophilic datasets further validate our approach's ability to address the challenging heterophily scenario through adaptive pathway balancing.

5.5. Discussion

The above comprehensive evaluation results demonstrate several key strengths of our proposed AG-GNN architecture. First, the model achieves remarkable consistency across diverse network settings, as evidenced by its superior average ranks of 1.33 and 1.17 on small-scale and large-scale networks, respectively. This consistency is particularly noteworthy given the varying challenges presented by different network structures. The performance improvement in heterophilic settings can be attributed to our model's adaptive gating mechanism. Traditional GNN architectures often struggle with heterophilic graphs due to their assumption that connected nodes should share similar features. Our adaptive approach effectively overcomes this limitation by dynamically adjusting the information flow based on both local and global graph properties. This is particularly evident in the results on Texas, Wisconsin, and Cornell, where AG-GNN significantly outperforms specialized heterophilic methods.

Furthermore, the model's ability to maintain performance at greater depths addresses a fundamental challenge in graph neural networks - the over-smoothing problem. The depth analysis reveals that AG-GNN can effectively utilize deeper architectures, particularly in heterophilic settings where additional layers help capture more complex structural patterns. This is in stark contrast to baseline models which typically suffer from severe performance degradation or complete failure at greater depths.

The scalability of AG-GNN is another crucial advantage. While several baseline models fail on larger graphs due to memory constraints, AG-GNN maintains both computational efficiency and superior performance. This scalability, combined with consistent performance across different network sizes, makes our model particularly suitable for real-world applications where graph scale and structure can vary significantly. Our experimental results also highlight the effectiveness of the parallel processing pathways in AG-GNN. The model's ability to adaptively combine information from different sources allows it to leverage both local and global graph properties effectively. This is particularly evident in the performance on networks with moderate homophily, where the model successfully balances the contributions of node features and structural information.

These findings validate our architectural design choices and demonstrate that AG-GNN effectively addresses several key challenges in graph representation learning: heterophily, over-smoothing, and scalability. The consistently superior performance across diverse settings suggests that our approach of combining adaptive gating with parallel information processing provides a robust foundation for graph neural network architectures.

6. Conclusion and future work

In this paper, we introduce AG-GNN, a novel graph neural network architecture that effectively addresses key challenges in graph representation learning: heterophily, over-smoothing, and scalability. Through an adaptive gating mechanism and parallel information processing pathways, our model demonstrates superior performance across a wide spectrum of graph structures and scales.

Our extensive experiments on 15 benchmark datasets, ranging from small-scale networks to graphs with millions of nodes, demonstrate the effectiveness and versatility of our approach. AG-GNN achieves state-of-the-art performance on both homophilic and heterophilic graphs, with particular strength in challenging heterophilic settings where traditional GNNs often fail. The model's ability to maintain performance at greater depths (up to 64 layers) represents a significant advance in addressing the over-smoothing problem that has long plagued deep graph neural architectures.

A key contribution of our work is the demonstration that effective handling of heterophilic graphs does not require sacrificing performance on homophilic structures. The adaptive nature of our gating mechanism allows the model to automatically adjust its behavior based on local and global graph properties, making it particularly suitable for real-world applications where graph characteristics may be unknown or mixed.

Several promising directions for future work emerge from our findings. First, investigating the application of our adaptive gating mechanism to other graph learning tasks, such as link prediction and graph classification, could yield valuable insights. Second, exploring the theoretical foundations of our model's stability in greater depth could provide guidelines for developing even more robust architectures. Finally, extending our approach to dynamic graphs and incorporating temporal information represents an important direction for handling evolving network structures.

The success of AG-GNN in handling both heterophilic and homophilic graphs while maintaining scalability suggests that adaptive mechanisms may be key to developing more general and robust graph neural architectures. As the field continues to evolve, we believe our work provides valuable insights for designing next-generation graph neural networks capable of handling the complexity and diversity of real-world graph data.

CRedit authorship contribution statement

Ahmed Begga: Writing – review & editing, Writing – original draft, Visualization, Validation, Software, Methodology, Investigation, Formal analysis. **Miguel Ángel Lozano:** Writing – review & editing, Writing – original draft, Investigation, Formal analysis. **Francisco Escolano:** Writing – review & editing, Writing – original draft, Methodology, Investigation, Funding acquisition.

Declaration of competing interest

The authors declare the following financial interests/personal relationships which may be considered as potential competing interests:

Francisco Escolano reports that financial support was provided by the Spanish Ministry of Science and Innovation. If there are other authors, they declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Acknowledgement

The authors acknowledge the support of the Spanish Government under project PID2022-142516OB-I00.

Data availability

No data were used for the research described in the article.

References

- [1] F. Scarselli, M. Gori, A.C. Tsoi, M. Hagenbuchner, G. Monfardini, The graph neural network model, *IEEE Trans. Neural Netw.* 20 (1) (2009) 61–80, <https://doi.org/10.1109/TNN.2008.2005605>
- [2] T.N. Kipf, M. Welling, Semi-supervised classification with graph convolutional networks, in: *International Conference on Learning Representations (ICLR)*, 2017.
- [3] Y. Xu, L. Han, T. Zhu, L. Sun, B. Du, W. Lv, Generic dynamic graph convolutional network for traffic flow forecasting, *Inf. Fusion* 100 (2023) 101946, <https://doi.org/10.1016/J.INFFUS.2023.101946>
- [4] H. Pei, B. Wei, K.-C.-C. Chang, Y. Lei, B. Yang, Geom-GCN: geometric graph convolutional networks, (2020).
- [5] T.N. Kipf, M. Welling, Variational graph auto-encoders, In *neurips Workshop On Bayesian Deep Learning* (2016) http://bayesiandeeplearning.org/2016/papers/BDL_16.pdf
- [6] Z. Wu, S. Pan, F. Chen, G. Long, C. Zhang, P.S. Yu, A comprehensive survey on graph neural networks, *IEEE Trans. Neural Netw. Learn. Syst.* 32 (1) (2021) 4–24, <https://doi.org/10.1109/TNNLS.2020.2978386>
- [7] J. Zhu, Y. Yan, L. Zhao, M. Heimann, L. Akoglu, D. Koutra, Beyond homophily in graph neural networks: current limitations and effective designs, in: *Proceedings of the 34th International Conference on Neural Information Processing Systems*, 2020.
- [8] M. Chen, Z. Wei, Z. Huang, B. Ding, Y. Li, Simple and deep graph convolutional networks, in: *Proceedings of the 37th International Conference on Machine Learning, ICLR 2020*, 13–18 July 2020, Virtual Event, vol. 119 of *PMLR*, 2020, pp. 1725–1735 *Proceedings of Machine Learning Research*. URL: <http://proceedings.mlr.press/v119/chen20v.html>
- [9] Y. Song, C. Zhou, X. Wang, Z. Lin, Ordered GNN: ordering message passing to deal with heterophily and over-smoothing, in: *The Eleventh International Conference on Learning Representations*, 2023.
- [10] T.K. Rusch, M.M. Bronstein, S. Mishra, A survey on oversmoothing in graph neural networks, *CoRR abs/2303.10993*, *arXiv:2303.10993*, <https://doi.org/10.48550/ARXIV.2303.10993>, 2023.
- [11] L. Zhao, L. Akoglu, Pairnorm: tackling oversmoothing in GNNs, (2019) <https://doi.org/10.48550/ARXIV.1909.12223>
- [12] Y. Rong, W. Huang, T. Xu, J. Huang, The truly deep graph convolutional networks for node classification, *CoRR abs/1907.10903*, *arXiv:1907.10903*, 2019 <http://arxiv.org/abs/1907.10903>
- [13] G. Li, M. Müller, A.K. Thabet, B. Ghanem, DeepGCNs: can GCNs go as deep as CNNs?, in: *2019 IEEE/CVF International Conference on Computer Vision, ICCV 2019*, Seoul, Korea (South), October 27 - November 2, 2019, IEEE, 2019, pp. 9266–9275, <https://doi.org/10.1109/ICCV.2019.00936>
- [14] P. Veličković, G. Cucurull, A. Casanova, A. Romero, P. Liò, Y. Bengio, Graph attention networks, *Int. Conf. Learn. Represent.* (2018).
- [15] J. Klicpera, A. Bojchevski, S. Günnemann, Personalized embedding propagation: combining neural networks on graphs with personalized pagerank, *CoRR abs/1810.05997* (*arXiv:1810.05997*, 2018) <http://arxiv.org/abs/1810.05997>
- [16] K. Karhadkar, P.K. Banerjee, G. Montúfar, FOSr: first-order spectral rewiring for addressing oversquashing in GNNs, in: *The Eleventh International Conference on Learning Representations, ICLR 2023*, Kigali, Rwanda, May 1–5, 2023, *OpenReview.net*, 2023, <https://openreview.net/forum?id=3YjQfCLdrzz>
- [17] Y. Yan, M. Hashemi, K. Swersky, Y. Yang, D. Koutra, Two sides of the same coin: heterophily and oversmoothing in graph convolutional neural networks, in: *IEEE International Conference on Data Mining, ICDM 2022*, IEEE, 2022.
- [18] D. Lim, F.M. Hohne, X. Li, S.L. Huang, V. Gupta, O.P. Bhalerao, S.-N. Lim, Large scale learning on non-homophilous graphs: new benchmarks and strong simple methods, in: *Advances in Neural Information Processing Systems*, 2021.
- [19] S. Luan, C. Hua, Q. Lu, J. Zhu, M. Zhao, S. Zhang, X.-W. Chang, D. Precup, Revisiting heterophily for graph neural networks, (2022) <https://doi.org/10.48550/ARXIV.2210.07606>
- [20] S. Abu-El-Haija, B. Perozzi, A. Kapoor, N. Alipourfard, K. Lerman, H. Harutyunyan, G. Ver Steeg, A. Galstyan, Mixhop: Higher-order graph convolutional architectures via sparsified neighborhood mixing, in: *International Conference on Machine Learning*, 2019.
- [21] Y. Li, D. Tarlow, M. Brockschmidt, R.S. Zemel, Gated graph sequence neural networks, in: Y. Bengio, Y. LeCun (Eds.), *4th International Conference on Learning Representations, ICLR 2016*, San Juan, Puerto Rico, May 2–4, 2016, Conference Track Proceedings, 2016, <http://arxiv.org/abs/1511.05493>
- [22] P. Barceló, E.V. Kostylev, M. Monet, J. Pérez, J. Reutter, J.P. Silva, The logical expressiveness of graph neural networks, in: *International Conference on Learning Representations*, 2020, <https://openreview.net/forum?id=r1LZ7AEKvB>
- [23] W. Hu, M. Fey, M. Zitnik, Y. Dong, H. Ren, B. Liu, M. Catasta, J. Leskovec, Open graph benchmark: datasets for machine learning on graphs, in: H. Larochelle, M. Ranzato, R. Hadsell, M. Balcan, H. Lin (Eds.), *Advances in Neural Information Processing Systems 33: Annual Conference on Neural Information Processing Systems 2020, NeurIPS 2020*, December 6–12, 2020, Virtual, 2020, <https://proceedings.neurips.cc/paper/2020/hash/fb60d411a5c5b72b2e7d3527cfc84fd0-Abstract.html>
- [24] J. Zhou, G. Cui, Z. Zhang, C. Yang, Z. Liu, M. Sun, Graph neural networks: a review of methods and applications, *CoRR abs/1812.08434*, *arXiv:1812.08434*, 2018) <http://arxiv.org/abs/1812.08434>
- [25] S. Xiao, S. Wang, Y. Dai, W. Guo, Graph neural networks in node classification: survey and evaluation, *Mach. Vis. Appl.* 33 (1) (2022) 4, <https://doi.org/10.1007/S00138-021-01251-0>
- [26] E. Chien, J. Peng, P. Li, O. Milenkovic, Adaptive universal generalized Pagerank graph neural network, in: *International Conference on Learning Representations*, 2021.

- [27] B. Xu, B. Xie, H. Shen, Towards deeper graph neural networks via layer-adaptive, in: Y. Ding, J. Tang, J.F. Sequeda, L. Aroyo, C. Castillo, G. (Eds.), Companion Proceedings of the ACM Web Conference 2023, WWW 2023, 30 April 2023 - 4 May 2023, 2023, pp. 103–106, Austin, TX, USA: ACM <https://doi.org/10.1145/3543873.3587323>
- [28] Y. Hou, J. Zhang, J. Cheng, K. Ma, R.T.B. Ma, H. Chen, M.-C. Yang, Measuring and improving the use of graph information in graph neural networks, in: International Conference on Learning Representations, 2020, <https://openreview.net/forum?id=rkelIkHKv5>
- [29] X. Li, R. Zhu, Y. Cheng, C. Shan, S. Luo, D. Li, W. Qian, Finding global homophily in graph neural networks when meeting heterophily, in: K. Chaudhuri, S. Jegelka, Le Song, C. Szepesvári, G. Niu, S. Sabato (Eds.), International Conference on Machine Learning, ICML 2022, 17–23 July 2022, Baltimore, Maryland, USA, Vol. 162 of Proceedings of Machine Learning Research, PMLR, 2022, pp. 13242–13256 URL:<https://proceedings.mlr.press/v162/li22ad.html>
- [30] F. Hoseinnia, M. Ghatte, M.H. Chehreghani, Mitigating over-smoothing in graph neural networks for node classification through adaptive early embedding and biased dropedge procedures, Knowl.-Based Syst. 320 (2025) 113615, <https://doi.org/10.1016/j.knosys.2025.113615>, <https://www.sciencedirect.com/science/article/pii/S0950705125006616>
- [31] Q. Li, Z. Han, X. Wu, Deeper insights into graph convolutional networks for semi-supervised learning, in: S. A. McIlraith, K. Q. Weinberger (Eds.), Proceedings of the Thirty-Second AAAI Conference on Artificial Intelligence (AAAI-18), the 30th Innovative Applications of Artificial Intelligence (IAAI-18), and the 8th AAAI Symposium on Educational Advances in Artificial Intelligence (EAAI-18), New Orleans, Louisiana, USA, February 2–7, 2018, AAAI Press, 2018, pp. 3538–3545, <https://doi.org/10.1609/AAAI.V32i1.11604>
- [32] N. Keriven, Not too little, not too much: a theoretical analysis of graph (over)smoothing, in: S. Koyejo, S. Mohamed, A. Agarwal, D. Belgrave, K. Cho, A. Oh (Eds.), Advances in Neural Information Processing Systems 35: Annual Conference on Neural Information Processing Systems 2022, NeurIPS 2022, New Orleans, LA, USA, November 28 - December 9, 2022, 2022, http://papers.nips.cc/paper_files/paper/2022/hash/0f956ca6f667c62e0f71511773c86a59-Abstract-Conference.html
- [33] X. Shen, Y. Zhang, Y. Xie, K. Wong, C. Peng, A block-based adaptive decoupling framework for graph neural networks, Entropy 24 (9) (2022) 1190, <https://doi.org/10.3390/E24091190>
- [34] W. Shen, M. Ye, W. Huang, Resisting over-smoothing in graph neural networks via dual-dimensional decoupling, in: J. Cai, M. S. Kankanhalli, B. Prabhakaran, S. Boll, R. Subramanian, L. Zheng, V. K. Singh, P. César, L. Xie, D. Xu (Eds.), Proceedings of the 32nd ACM International Conference on Multimedia, MM 2024, Melbourne, VIC, Australia, 28 October 2024 - 1 November 2024, ACM, 2024, pp. 5800–5809, <https://doi.org/10.1145/3664647.3681204>
- [35] Q. Cheng, L. Long, J. Xu, M. Zhang, S. Han, C. Zhao, W. Feng, A universal strategy for smoothing deceleration in deep graph neural networks, Neural Networks 185 (2025) 107132, <https://doi.org/10.1016/j.neunet.2025.107132>, <https://www.sciencedirect.com/science/article/pii/S0893608025000115>
- [36] J. Zhou, Y. Du, R. Zhang, J. Xia, Z. Yu, Z. Zang, D. Jin, C. Yang, R. Zhang, S.Z. Li, Deep graph neural networks via posteriori-sampling-based node-adaptive residual module, in: A. Globerson, L. Mackey, D. Belgrave, A. Fan, U. Paquet, J. Tomczak, C. Zhang (Eds.), Advances in Neural Information Processing Systems, vol. 37, Curran Associates, Inc., 2024, pp. 68211–68238, https://proceedings.neurips.cc/paper_files/paper/2024/file/7e0dc9ccba0f1333be13a3f9dc2b3138-Paper-Conference.pdf
- [37] S.K. Maurya, X. Liu, T. Murata, Improving graph neural networks with simple architecture design, (2021).
- [38] M. Zou, Z. Gan, R. Cao, C. Guan, S. Leng, Similarity-navigated graph neural networks for node classification, Inf. Sci. 633 (2023) 41–69, <https://doi.org/10.1016/J.INS.2023.03.057>
- [39] W.L. Hamilton, R. Ying, J. Leskovec, Inductive representation learning on large graphs, in: Proceedings of the 31st International Conference on Neural Information Processing Systems, 2017.
- [40] X. Wang, M. Zhang, How powerful are spectral graph neural networks, in: K. Chaudhuri, S. Jegelka, Le Song, C. Szepesvári, G. Niu, S. Sabato (Eds.), International Conference on Machine Learning, ICML 2022, 17–23 July 2022, Baltimore, Maryland, USA, vol. 162 of PMLR, 2022, pp. 23341–23362 Proceedings of Machine Learning Research. URL:<https://proceedings.mlr.press/v162/wang22am.html>
- [41] Y. Wu, Y. Wang, L. Hu, J. Hu, DCGNN: Adaptive deep graph convolution for heterophily graphs, Inf. Sci. 666 (2024) 120427, <https://doi.org/10.1016/J.INS.2024.120427>
- [42] P. Sen, G. Namata, M. Bilgic, L. Getoor, B. Galligher, T. Eliassi-Rad, Collective classification in network data, AI Mag. 29 (3) (2008) 93, <https://doi.org/10.1609/aimag.v29i3.2157>, <https://ojs.aaai.org/index.php/aimagazine/article/view/2157>
- [43] S. Chanpuriya, C.N. Musco, Simplified graph convolution with heterophily, in: A. H. Oh, A. Agarwal, D. Belgrave, K. Cho (Eds.), Advances in Neural Information Processing Systems, 2022, <https://openreview.net/forum?id=jRpiqxrWm>
- [44] X. Li, R. Zhu, Y. Cheng, C. Shan, S. Luo, D. Li, W. Qian, Finding global homophily in graph neural networks when meeting heterophily, in: K. Chaudhuri, S. Jegelka, Le Song, C. Szepesvári, G. Niu, S. Sabato (Eds.), International Conference on Machine Learning, ICML 2022, 17–23 July 2022, Baltimore, Maryland, USA, vol. 162 of PMLR, 2022, pp. 13242–13256 Proceedings of Machine Learning Research. URL:<https://proceedings.mlr.press/v162/li22ad.html>